Flight Right

A speaker's briefing

What we built, what is left, and what changed along the way — written for someone presenting this who does not write code and does not work in aviation.

> **If you remember one sentence, remember this one.**
>
> Flight Right watches aircraft near Mumbai, predicts which pairs are going to come dangerously close, and tells the air traffic controller early enough to do something about it — **with at least three minutes' notice, 94% of the time.** It suggests what to do. It never does it. A human always decides.

Status as of 7 September 2026. Every number in this document was measured on real recorded air traffic that the models had never seen during training. Companion document: *evaluation.md* has the full technical detail.

1. The five words you need

Everything else in this briefing is built out of these. If an examiner uses one of these words, this is what they mean.

Word	What it means
ATC / controller	Air Traffic Control. A person in a room watching a radar screen, telling pilots where to fly. They are the customer for everything we built.
Separation	The legally required empty space between two aircraft. Away from airports it is **5 nautical miles apart sideways**, or **1,000 feet apart in height**. Either one is enough — aircraft stacked 1,000 ft apart may pass directly over each other.
Conflict	Two aircraft that are going to break that rule if nobody intervenes. Note: not a crash. A conflict is a rule being broken, which happens far more often and is exactly what controllers spend their day preventing.
Closest approach	The moment in the future when two aircraft will be nearest to each other. Predicting *when* and *how close* is the core of the whole system.
ADS-B	The radio signal each aircraft broadcasts saying where it is, how high, how fast, which way. Free to receive. It is our only input — we get positions, and nothing else.

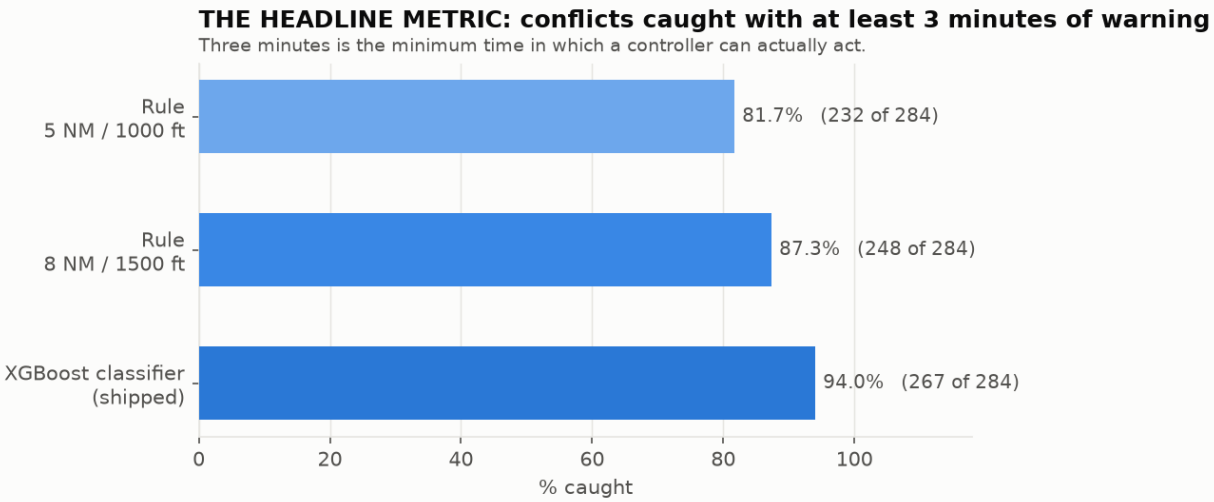
The one thing that explains most of our design decisions

We know where every aircraft is. We do not know where it has been told to go. A real control centre has the flight plan and the instructions the controller just issued. We have only the position broadcast. So when an aircraft that has been flying level for two minutes suddenly starts descending, we did not see that coming, because the reason lives in a radio conversation we cannot hear.

This single limitation explains why we predict several possible futures instead of one, why one obvious improvement failed, and why we always say this is a prototype rather than a certified system. If you are asked what the biggest weakness is, **this is the honest answer**, and giving it makes us look careful rather than naive.

2. The headline number

Lead with this. Everything else is supporting detail.



Measured on 284 real conflicts in held-out test data the models never saw during training.

Why three minutes, and not some other number?

Because that is how long it physically takes to change anything. Work it through: the controller has to notice and decide (about 15 seconds), call the pilot on the radio and hear it read back (about 15 seconds), and the pilot has to react (about 8 seconds). That is **38 seconds before the aircraft moves at all**. Then a 1,000-foot climb takes roughly 40 seconds, and a turn takes roughly 96 seconds. Add margin and you get three minutes.

We then checked this a completely different way. We simulated every instruction a controller could give to a pair of aircraft 90 seconds from disaster — **all 26 of them fail.** There is no climb, no turn and no speed change that opens a safe gap in the time available. Two independent methods, same answer. That is a strong thing to say out loud.

Say it this way

Say this	Not this
“We catch 94% of conflicts with at least three minutes of warning.”	“Our model is 94% accurate.” — a different and much weaker claim
“An AI-assisted decision-support prototype, with a human always in the loop.”	“An air traffic control system.” — we are not certified and must not imply it
“The simple geometric method catches 87%; ours catches 94% on the same data.”	“Our AI beats the traditional approach.” — vague, and invites “by how much?”

3. What we have actually built

Five pieces. Each one does a job the one before it cannot.

Model 1 — Where will this aircraft be?

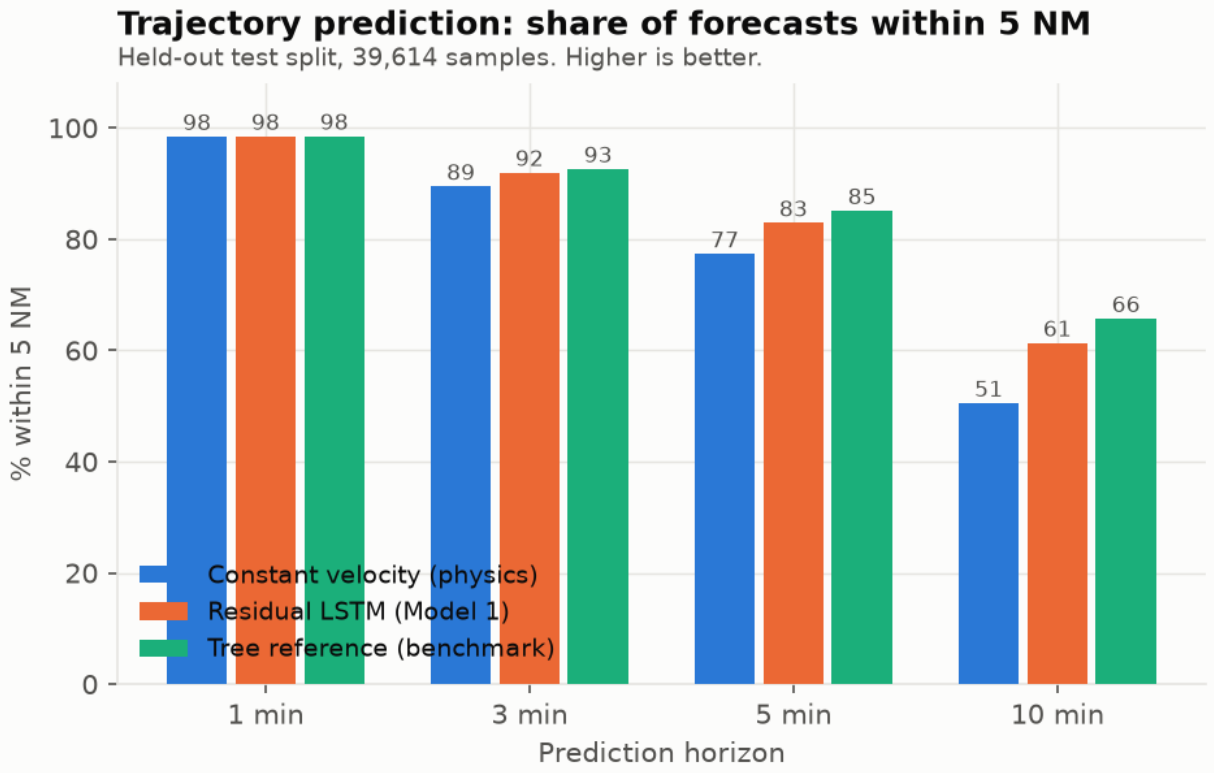
The job: given the last ten position reports from one aircraft, predict where it will be in 1, 3, 5 and 10 minutes.

How it works, without the jargon: the obvious method is to draw a straight line — it is going that way at that speed, so in ten minutes it will be there. That works surprisingly well and it is our baseline. Our model does **not** replace it. Instead it learns to answer a narrower question: *how wrong is that straight line likely to be, and in which direction?* It then nudges the straight-line answer.

Why that is clever, and worth saying: if the model has nothing useful to add, it outputs zero and we get the straight-line answer back exactly. **It is structurally incapable of being worse than plain arithmetic.** The first version of this model did not work that way, and it was worse than arithmetic for three weeks without anyone noticing.

Result: at ten minutes, the straight line lands within 5 nautical miles 50.5% of the time; ours does 61.3%.

> **Be ready for this question: "is this your best model?"** Honestly, no. A much simpler, non-AI method scores 65.7% where our neural network scores 61.3%. We keep both, we report both, and we say so. Being caught hiding that would be far worse than admitting it. It also is not embarrassing: it tells us the limit is the *information available*, not the cleverness of the model — and we know exactly what information is missing (the controller's instructions).



Higher is better. Notice the three bars are nearly identical at 1 minute and spread apart at 10 minutes — predicting further ahead is where the difficulty lives.

Model 1b — Several futures instead of one

The job: offer five possible paths, each with a probability, instead of one confident guess.

Why: we measured something uncomfortable. Take two aircraft in a genuinely identical situation — same position, same heading, same speed, same altitude. Ten minutes later they are, on average, **7.9 nautical miles apart**. The future is not merely hard to predict; it is **genuinely undetermined** by what we can see, because a controller's instruction decides it and we never hear that instruction.

So we stopped guessing and started covering. For spotting conflicts we never have to pick one path — we check all five, and if any of them collides we raise the warning. Safety comes from covering the possibilities, not from guessing right.

Result: for arriving aircraft at ten minutes, one of the five paths is within 5 miles 63.6% of the time, against 15.8% for the straight line.

> **An honesty trap to avoid.** That 63.6% means *one of the five* was right. If you are asked how often the *single most likely* one is right, the answer is 38.6%. Both numbers are true and they measure different things. Quoting 63.6% as “accuracy” would be wrong, and an examiner who knows the field will spot it.

Model 2 — Will these two aircraft actually get too close?

The job: look at a *pair* of aircraft and decide whether they will really lose separation.

Why we need it at all, given we can already do the geometry: because the straight-line prediction is wrong in a specific, learnable way. When it misses a real conflict, **99% of the time the mistake is vertical**. It correctly sees the two aircraft passing close side-to-side, but it thinks they will be about 2,825 feet apart in height — and they actually end up about 498 feet apart. The reason is that one of them is climbing or descending about eight times faster than normal, and a straight-line guess handles that badly.

So the model's real job is to learn where our own predictions go wrong. That is a genuinely useful thing that is available no other way, and it is a good answer to “why not just use the maths?”

Result: it catches 94.4% of real conflicts where the geometric rule catches 88.1%. Tested at the *same* false-alarm cost, so it is not simply crying wolf more often.

Model 3 — What should the controller actually do?

The job: for a confirmed alarm, propose up to three ways out — change altitude, change heading, or change speed — each with a plain sentence a controller could read aloud, an estimated cost in delay and fuel, and a confidence figure.

The design decision worth explaining: the lazy version would teleport the aircraft to its new heading and check whether that fixes things. That would confidently recommend turns no real aircraft could complete in time — wrong in the dangerous direction. Instead we **fly every suggestion in simulation**: for the first 38 seconds nothing happens at all (decision, radio call, pilot reaction), and only then does the aircraft start moving, at the speed a real airliner actually turns and climbs.

And it checks the whole picture, not just the two aircraft. A real example from our tests: two aircraft are head-on, and the obvious fix is to descend one of them. That works — and puts it head-on into a third aircraft 2,000 feet below, losing separation 79 seconds later. The system rejects that suggestion and names the aircraft it would have endangered. This is a known problem in air traffic research called the **domino effect**; our test reproduced it on the first attempt.

Nothing is ever applied automatically. The system offers; the controller accepts, dismisses or overrides; the decision is logged. Say this clearly — it is a safety and ethics point, and it will be asked about.

The alert feed — what the controller actually sees

The job: stop drowning the controller.

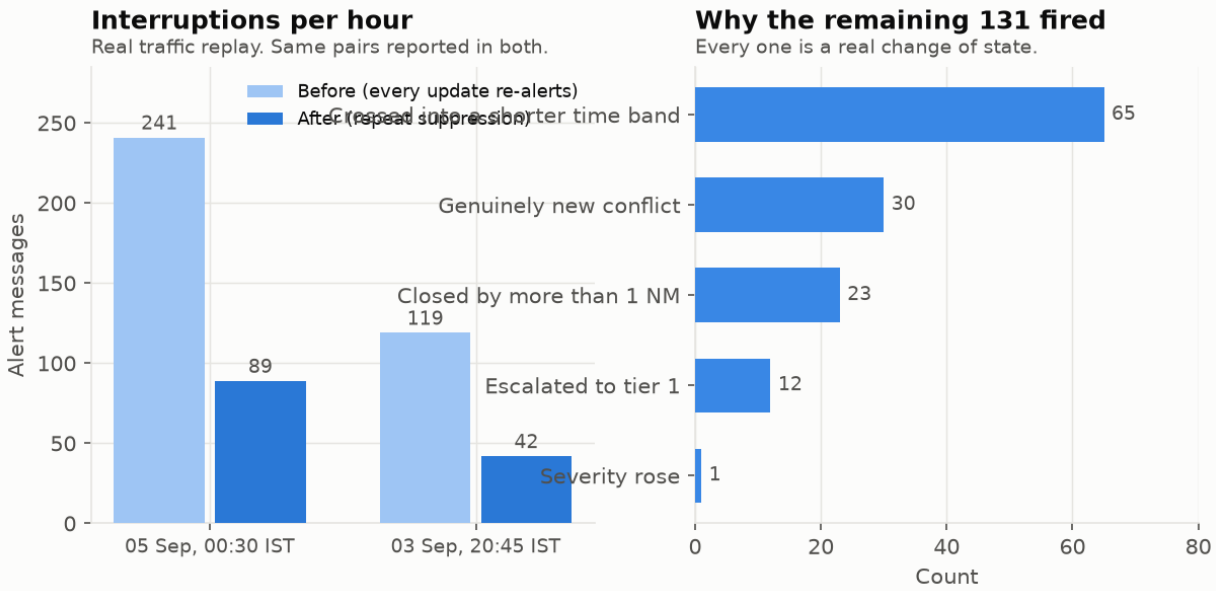
The problem we found by counting: the system re-examined every pair of aircraft every ten seconds with no memory of what it had already said. A pair converging for four minutes generated an alert every single update — about twenty-four alerts for one situation the controller understood after the first. One hour of real Mumbai traffic produced **297 alert messages for 30 actual situations.**

What we built: a memory. A situation already on screen does not interrupt again unless something genuinely gets worse — it moves into the three-minute danger band, it gets more severe, the suggested manoeuvres change or run out, or the aircraft close significantly. Separately, the least confident items are gathered into one expandable line instead of taking a row each.

The safety guarantee, and how to say it.

This is the part to be careful about, because “we show fewer warnings” sounds like “we hide some conflicts”. It is not. **Every confirmed conflict stays on the screen, permanently, without exception.** What we changed is whether it makes a *noise* again, not whether it is *there*. Those are different things.

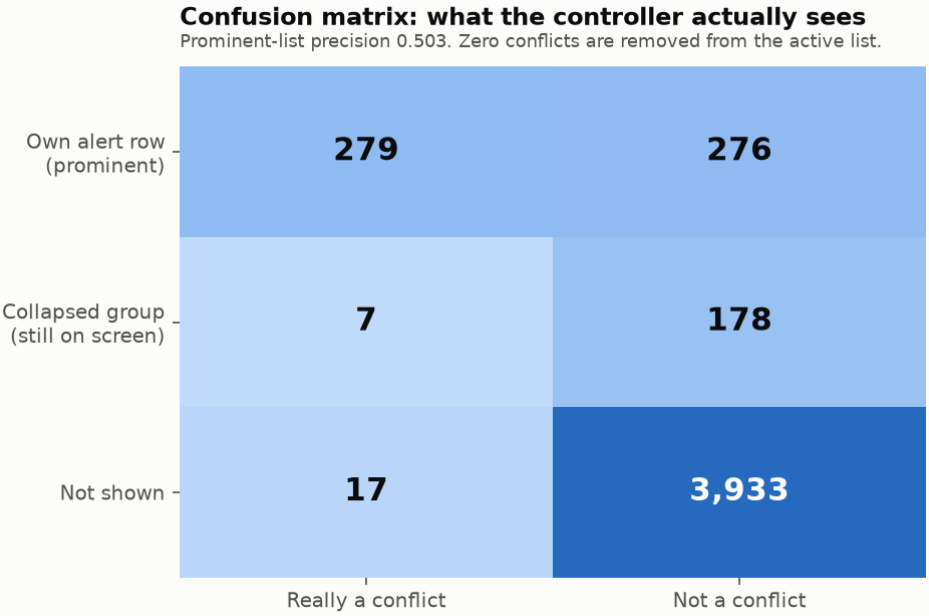
We proved it rather than claiming it: we replayed two hours of real traffic twice, once with the feature and once without, and compared the lists of aircraft pairs reported. **Identical — 30 pairs both times, zero lost.** Interruptions fell by 64%.



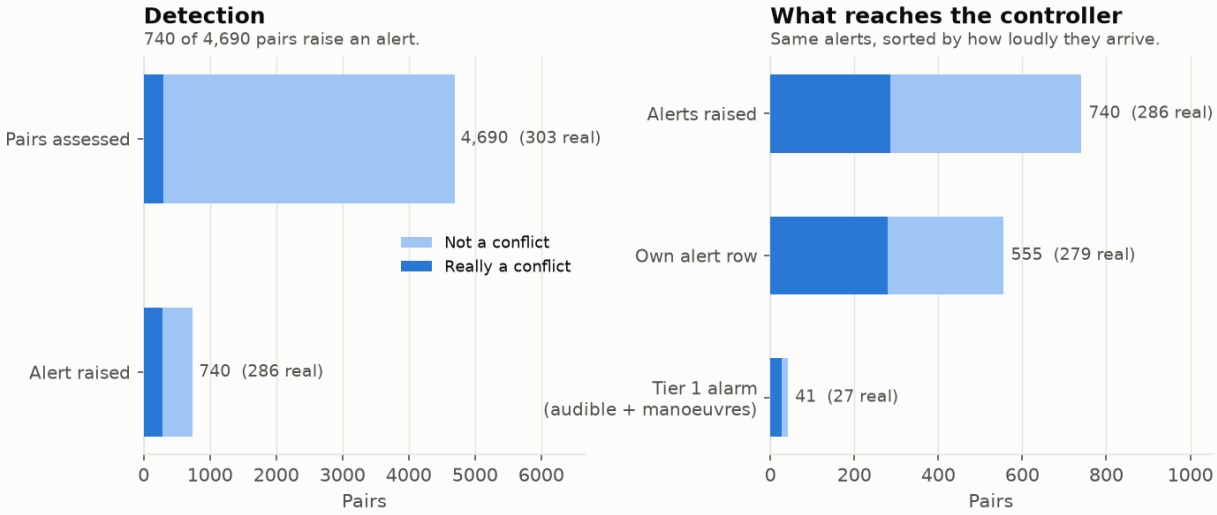
Left: far fewer interruptions. Right: every interruption that remains is a genuine change in the situation, not a repeat.

4. What the controller sees, in numbers

This is the section most likely to be probed, because the raw figure sounds bad and the real picture is much better. Learn this page.



Read across the rows. Of 4,690 aircraft pairs examined, 303 were genuinely going to lose separation.



Left: of 4,690 pairs, 740 raise a warning. Right: of those 740, only 41 become an audible alarm with suggested manoeuvres.

The number that sounds bad

About **six in ten warnings are for situations that never actually become dangerous**. Do not hide this — say it first, then explain it. Three things make it far less alarming than it sounds:

- **Almost all of them are silent.** The system has two levels. A **tier 1 alarm** means “act now”, makes a noise, and comes with suggested manoeuvres. A **tier 2 entry** is a quiet line on a watch list for aircraft still five to ten minutes apart. **97% of the false warnings are quiet watch-list entries.** Of the actual alarms, about two thirds are real.

- **Most “false” warnings are not aircraft in different parts of the sky.** The typical one passes about 2.4 miles apart sideways — well inside the 5-mile rule — but stays 1,930 feet apart in height, so it stays legal. The system saw a genuinely tight situation and was right to look.
- **Missing a real conflict is far worse than a false alarm.** A false alarm costs a controller a glance. A missed conflict is the thing the system exists to prevent. We tuned deliberately towards catching more, and we can defend that trade.

5. What we said we would build, and what we actually built

This is the most valuable section for questions. Every row is a case where the original plan was changed because we measured something. **Being able to explain a change is stronger than having had a perfect plan.** Nothing here was changed on a hunch.

Originally planned	What we built instead	Why — the measurement
A neural network that reads the flight path forwards *and backwards*, and predicts the aircraft's position directly.	A network that reads forwards only, and predicts *how wrong the simple straight-line guess is*.	The original was worse than plain arithmetic at every horizon for three weeks. It could see where an aircraft was but not which way it was going. Reading backwards also makes no sense — at the moment of prediction there is no future to read.
Score the model by average squared error (the standard textbook choice).	Score it by a measure that does not over-punish the few worst cases.	The textbook choice chases a handful of extreme errors and ruins thousands of near-perfect predictions. Near-perfect predictions fell from 27.7% to 6.4% — worse than doing nothing at all.
One conflict check looking ten minutes ahead.	Two levels: an **alarm** for 0–3 minutes that suggests manoeuvres, and a silent **watch list** for 3–10 minutes that never does.	A ten-minute prediction is not accurate enough to justify turning a real aircraft. Turning an aircraft that was never going to conflict costs real fuel and real delay.
Check the gap between aircraft at four moments: 1, 3, 5 and 10 minutes.	Check it every 15 seconds.	Two aircraft closest at four minutes were measured at three and five minutes — where they were far apart — and the conflict vanished. **21% of real conflicts were invisible** to the four-moment check.
Raise an alert when aircraft break the legal minimum.	Raise it on a deliberately wider volume.	At the bare legal minimum we caught only 46% of real conflicts, because a prediction one mile out flips the answer. At the wider setting we catch 99%.
Train the conflict detector on invented practice scenarios.	Rebuilt from scratch: both aircraft real, only the *introduction* between them staged.	The original was circular — it was given the answer as one of its inputs. We verified this: the answer could be recalculated from its own inputs for **40,000 rows out of 40,000**. It scored perfectly while having learned nothing.
Practice conflicts drawn at random.	Only conflicts that *develop* while you watch.	A quarter of the practice conflicts had the two aircraft already at their closest point at the first moment we saw them. There is no warning to give, so measuring warning time on them was meaningless. Now 0%.

Originally planned	What we built instead	Why — the measurement
“Prefer altitude changes when time is short”, written into the specification as a physics fact.	Test every option on every conflict, and reject the ones that do not work.	The arithmetic behind the rule is right but the conclusion is not general. We tested it: climbs and turns become available at exactly the same moment. Altitude still usually ranks first, but for a different reason — it is easier for a controller to manage.
Check a suggestion against the other aircraft, assumed to fly straight.	Check it against that aircraft's five possible futures *and* every other aircraft nearby.	The old version offered a descent that solved one conflict and created another 79 seconds later with a third aircraft. It is a named problem in the research literature and our test reproduced it immediately.
Add artificial radar noise during training to make the model robust.	Dropped.	Our data is not radar — it is the aircraft's own broadcast of its position. We also have no measurement of Indian radar noise to base an injection on, so it would have been inventing a problem and then solving it.
Cite five published papers to justify the design.	Justify the design by our own measurements, and cite papers that are actually about this problem.	We read all five in full. **None of them proposes or tests a flight-path prediction model.** Two are about converting radio speech to text. If asked where the design comes from, the honest and stronger answer is: we designed it, and here is the measurement.

6. One thing we tried that failed

Worth telling deliberately. It shows we test our own ideas rather than only reporting the ones that worked.

The idea. Nearly two thirds of our false alarms are aircraft that pass close side-to-side but stay comfortably stacked in height — perfectly normal traffic at different altitudes. So: if both aircraft are flying level and there is already more than 1,000 feet between them, say nothing. It sounds obviously right.

It works, and we cannot use it. It removes 46% of false alarms — and throws away **21 real conflicts**. Our headline number would fall from 94.0% to 86.6%.

Why it fails, which is the interesting part. Those 21 aircraft *were* flying level when we looked. Then one of them started descending. Several were predicted to pass more than 2,000 feet apart and actually ended up 240 to 530 feet apart. The descent came from a controller's instruction that we never see.

> **The line that turns a failure into a finding.**
>
> Real air traffic systems *do* use exactly this filter. Europe's official guidance lists it — and warns in the same sentence that it “reduces warning time in the event of level bust”, which is precisely what happened to us. They get away with it because their system knows what altitude each aircraft has been *cleared* to fly at. We are guessing from whether the aircraft happens to be climbing right now, and that guess is exactly what fails.
>
> So this is not just a failed experiment. It is a **concrete, measured demonstration of why a production version would need flight-plan data** — which is far more convincing than a generic disclaimer.

7. What is still left to build

Be straightforward about this. The intelligence is built and measured; the plumbing that connects it to a live screen is not.

Piece	Status	What it means in plain terms
The four models	**Done**	Prediction, multiple futures, conflict detection and suggested manoeuvres all work and are measured.
The alert feed	**Done**	Controls what reaches the screen and how often it interrupts.
Automated checking	**Done**	110 automatic checks run in about a minute and confirm nothing has broken.
The web server	**Not started**	The piece that sits between the models and the screen. The models can be loaded and run; there is no service wrapping them yet.
The database	**Not started**	Where aircraft history, conflicts and the record of every controller decision get stored.
Connecting the dashboard	**Not started**	The screen exists and works, but on pretend data. It has never been connected to the real models.
Audit logging on the server	**Not started**	Currently the record of decisions lives only in the browser, which is demonstration-grade, not the real thing.

Piece	Status	What it means in plain terms
One threshold decision	**Waiting on us**	How eager the system should be to raise alarms. We have measured the options and their costs; choosing changes when a real controller is interrupted, so it is a human decision, not a technical one.

How to answer “so it does not actually work yet?”

> “The decision-making works and is measured on real recorded traffic — that is the hard part and it is done. What is not built is the live plumbing: the server and database that would feed it real-time data and store the controller's decisions. That is engineering work with no research risk left in it.”

8. Questions you will be asked

“Is this real AI, or just maths?”

Both, deliberately. There is a simple geometric calculation underneath that always runs, and machine learning on top of it that corrects its known blind spots. We never removed the simple version — if a model fails, the geometry still catches conflicts. Rules alone are not smart enough; AI alone is not safe enough.

“Could this control aircraft?”

No, and deliberately not. It only ever suggests. A controller accepts, dismisses or overrides every suggestion, and the decision is recorded. We would not build it otherwise.

“Why Mumbai?”

It is one of India's busiest and most constrained airports — dense mixed traffic, strong monsoon weather, runway sequencing pressure. It gives a demonstrable story without needing certified national air traffic data, which we could not get.

“Where did your training data come from?”

Twelve days of real aircraft movements around Mumbai, collected from a free public network, about 627,000 individual position reports. Real losses of separation are rare by design, so we could not wait for them — we take two *real* aircraft and stage an introduction between them, sliding one in time and space so their paths cross. Every turn, climb and speed change in both is real. **We never invent flight paths**, because a model trained on invented flying learns our assumptions rather than Mumbai's traffic.

“Does it work near the airport, where aircraft are closest?”

Not yet, and that is a deliberate limit rather than an oversight. The 5-mile rule is for open sky; near an airport the legal minimum drops to 3 miles, and closer still on final approach. We measured what happens if you apply the open-sky rule everywhere: it flags something in over 90% of all moments, because normal landing traffic is legitimately close together. So we restricted the system to aircraft above 10,000 feet. Handling the airport area properly needs its own separate rules and its own training data.

“What is the biggest weakness?”

That we can see where aircraft are but not where they have been told to go. Everything we cannot predict traces back to that. It is also why we are honest that a production version would need certified radar, flight plans, weather and regulatory approval — none of which a student project can obtain.

“How do you know the numbers are real?”

Three things. We always test on later days than we trained on, never a random shuffle — aircraft positions seconds apart are nearly identical, so a random split would let the model see its own answers. We always print the simple non-AI baseline beside every result on the same data. And we run 110 automatic checks that verify the saved models from scratch, including one that specifically re-tests for the “given the answer” mistake we made once already.

“What would you do with more time?”

Finish the server and connect the screen. Then get flight-plan data, which is the single change that would improve the results most — it would let us use the filter that currently fails, and it would resolve

most of what we cannot predict.

> **Three habits that make the whole thing land better.**
>
> **1. Lead with the 94%, then explain what it means.** Do not open with technical detail.
>
> **2. Volunteer the weaknesses before you are asked.** The tree that beats our neural network, the six-in-ten false alarms, the failed filter. Every one of them has a good explanation, and offering them first reads as rigour rather than as being caught out.
>
> **3. Never claim it is a working air traffic control system.** Prototype, decision support, human always in the loop. Every time.